data replicable and consistent. You can do this with a combination of encapsulation and test doubles. You encapsulate the data by creating a service object that wraps around the changing functionality. By mediating access to the changing functionality, you make it easier to stub or mock the output values. Stubbing the values provides the consistency you need for testing—you provide the exact value that will be used in the test.

You might create a RandomStream class that wraps Ruby's rand() method:

```
class RandomStream
  def next
    rand()
  end
end
```

This example is a little oversimplified—normally you'd be encapsulating RandomStream. With your own wrapper class, you can provide more specific methods tuned to your use case—something like def random_phone_number. First you unit-test the stream class to verify that the class works as expected. Then any class that uses RandomStream can be provided mock random values to allow for easier and more stable testing.

The exact mix of encapsulation and mocking varies. The Rails ActiveSupport gem has methods that stub the time and date classes with no encapsulation, allowing you to specify an exact value for the current time for testing purposes.

In Chapter 14, *Testing External Services*, on page 285, I'll discuss this pattern for wrapping a potentially variable external service in more detail. I'll cover mock objects in Chapter 7, *Using Test Doubles as Mocks and Stubs*, on page 129, and I'll talk more about debugging intermittent test failures in Chapter 15, *Troubleshooting and Debugging*, on page 303.

Independent

A test is *independent* if it doesn't depend on any other tests or external data to run. An independent test suite gives the same results no matter the order in which the tests are run, and tends to limit the scope of test failures to only tests that cover a buggy method.

In contrast, a very dependent test suite could trigger failures throughout your tests from a single change in one part of an application. A clear sign that your tests are not independent is if you have test failures that happen only when the test suite is run in a particular order—in fully independent tests, the order in which they are run should not matter. Another sign is a single line of code breaking multiple tests. You can also uncomment the generated line